Delft University of Technology
Faculty of Electrical Engineering, Mathematics, and Computer Science

DSAIT4110 Sequential Decision Making

Assignment 2: Model Predictive Control (MPC)

Lecturer: Anahita Jamshidnejad

February 23, 2026 (2025–2026 Q3)

This assignment complements the theoretical content presented in Lectures 4–6:

- State-space prediction, stability, and discrete-time dynamics
- Model Predictive Control (MPC): optimization, cost functions, constraints
- Robust Tube-based Model Predictive control (Tube MPC): nominal versus actual trajectories, disturbance handling

You will work with simulation templates provided by the instructor. The goal is to deepen conceptual understanding, not coding from scratch.

Provided scripts: To support the lab session, we provide MATLAB scripts containing all core simulation code for the linear MPC controller, prediction model, and Tube MPC visualization. Students are expected to modify the parameters only (e.g., horizons, cost weights, feedback gains, disturbance bounds), not to implement the algorithms from scratch. If preferred, students may use the provided MATLAB scripts as a reference and translate them into Python or another programming language of their choice; however, this is entirely optional and not required for the assignment.

Module 1: Prediction and system behavior (open loop versus closed loop)

Learning objectives

- Understand multi-step prediction for discrete-time linear systems.
- Observe how disturbances affect open-loop trajectories and why this motivates closed-loop control.
- Connect observations to stability concepts from Lecture 4.

Background

We consider the discrete-time linear system

$$\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k + \boldsymbol{d}_k, \tag{1}$$

as used in Lecture 4 (and 5) for prediction and stability intuition. We have included an additive disturbance $\boldsymbol{d}_k \in \mathbb{R}^2$ to the state. We assume that the state vector belongs to $\mathbb{R}^2$ and $u \in \mathbb{R}$ is a scalar value.

What you are asked to change in the script

- Initial state $\boldsymbol{x}_0$ (given as `x0` in the script)
- Constant scalar input u (given as `u` in the script)
- Disturbance time step k (given as `disturbance_step` in the script)
- Disturbance vector $\boldsymbol{d}$ (given as `d` in the script)

Feel free to run additional trials changing other parameters and values (e.g., dynamic and input matrices).

Tasks (run $\rightarrow$ modify $\rightarrow$ analyze)

1. **Base run (open-loop prediction):** Run the script with the default parameters and values (no disturbance `d = [0; 0]`). It generates a plot for the open-loop predicted state trajectories x_1 and x_2 versus time step k. Save the plot and use it, when relevant, in the following analyses.

2. **Effect of initial state:** Change x_0 (given by `x0 = [-2; 3]` as default value in the script) to at least two different values (e.g., `x0 = [-1;1]` and `x0 = [2;-3]`), re-run and save the plots to use them for the next analysis.
   Analyze: How do the trajectories differ? Which state component is most sensitive to your changes and why (use the structure of dynamic and input matrices A and B for your explanation)?

3. **Effect of constant input:** Vary the constant control input u, which by default is given as `u = 0.1` in the script. Try $u \in \{0,\ 0.05,\ 0.2\}$ with the same initial state $\boldsymbol{x}_0$ for all cases. You will receive three more plots; save them and use them for the next analysis.
   Analyze: How does u bias the motion over time? How is this visible in trajectories of x_1 and x_2?

4. **Disturbance effect:** Fix $\boldsymbol{x}_0$ and u, and inject non-zero disturbances. Vary the disturbance such that $\|d\| \in \{0.2,\ 0.5,\ 1.0\}$ (choose direction freely) and disturbance injection time according to `disturbance_step`$\in \{10,\ 15,\ 20,\ 30\}$.
   Analyze: Why and how do actual states deviate from the open-loop predicted ones when a disturbance occurs? Relate your answer to model mismatch and the absence of feedback.

5. **Closed-loop preview:** Enable the simple state-feedback (closed-loop) input at the end of the script, run, and save your generated plots.
   Analyze: Compare the closed-loop plots with corresponding open-loop ones and explain how system behavior and state convergence are affected.

Module 2: MPC and horizon effects

Learning objectives

- Reinforce the MPC loop: predict $\rightarrow$ optimize $\rightarrow$ apply $\rightarrow$ repeat.
- Understand the effect of the prediction horizon size N^{P} on closed-loop behavior.
- Investigate how cost weights Q and R shape performance and input smoothness.
- Connect the observed behavior with Lecture 5 (constraints, cost shaping, receding horizon).

Background

At every decision-making step k, MPC solves $\min_{\boldsymbol{u}_k,\ldots,\boldsymbol{u}_{k+N^{\mathrm{P}}-1}} \sum_{i=0}^{N^{\mathrm{P}}-1} \ell(\boldsymbol{x}_{k+i}, \boldsymbol{u}_{k+i}) \; + \; \ell_f(\boldsymbol{x}_{k+N^{\mathrm{P}}})$, subject to system dynamics and constraints. Only the first input $\boldsymbol{u}_k$ is applied; output is measured; then the optimization is repeated at $k+1$.

The MATLAB script implements the same discrete-time linear system used in Module 1 without injecting disturbances $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$ with $A = \begin{bmatrix} 1 & 0.1 \\ 0 & 1 \end{bmatrix}$, $B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. It considers input constraint $-0.8 \leq u_{k+i} \leq 0.8$ for all time steps.

What you are asked to change in the script

- Prediction horizon N^{P} (given as `Np` in the script).
- Cost weights: state-weight matrix Q and input-weight matrix R (in the script as `Q` and `R`).

Tasks (run $\rightarrow$ modify $\rightarrow$ analyze)

1. **Base run (short horizon):** Set `Np = 3` in the script and run the closed-loop simulation. Save the plots of state trajectories, control input sequence, any constraint activity indicators. These plots will serve as a reference for the following steps.

2. **Effect of horizon length:** Increase the prediction horizon to `Np = 10` and then to `Np = 30`. Keep the same initial state, constraints, and cost weights. You may try this for more variations of N^{P} to get a more extensive view of the impact of prediction horizon size on system behavior.
   Analyze: Compare the results for the three simulated prediction horizons in terms of:
   - convergence rate of the closed-loop trajectory,
   - smoothness of the control input,
   - activation of state or input constraints.

   Explain *why* larger horizons change the behavior (*hint:* relate to "seeing further into the future").

3. **Effect of cost weights:** Fix the value of the prediction horizon (e.g., at 10) and modify the cost weights. Increase Q (e.g., multiply by 5) and keep R fixed. Save the corresponding plots.
   Increase R (e.g., multiply by 5) and keep Q fixed. Save the corresponding plots.
   Analyze:
   - Why does increasing Q result in more aggressive state regulation?
   - Why does increasing R produce smoother and more conservative inputs?
   - How do these observations match the quadratic cost expression from Lecture 5?

4. **Summary reflection:** Using all your observations, summarize in one paragraph: (1) how the prediction horizon length shapes MPC behavior, and (2) how cost weights Q and R influence the trade-off between fast convergence and smooth actuation. Connect your conclusions to the role of prediction horizon, cost shaping, and constraints discussed in Lecture 5.

5. **Effect of terminal cost (value shaping):** Activate the indicated piece of code in the provided MATLAB script. This enables a terminal cost $\ell^{\mathrm{f}}\left(\boldsymbol{x}_{k+N^{\mathrm{P}}}\right) = \boldsymbol{x}_{k+N^{\mathrm{P}}}^{\top} P \, \boldsymbol{x}_{k+N^{\mathrm{P}}}$ with P computed via the script as the solution of the Discrete-time Algebraic Riccati Equation (DARE) for (A, B, Q, R) (i.e., the LQR terminal weight). Keep the same prediction horizon size and the same Q and R weights as in your previous runs. Note that you may need to run this experiment for several values of N^{P} (from small to large) to observe how the prediction horizon determines the impact of the terminal cost.

   Analyze: Compare the resulting trajectories and inputs against the case where no terminal cost was used (with same N^{P} and Q, R). Discuss the following questions:
   - Whether the first one or two inputs become less aggressive (e.g., fewer early saturations).
   - How the terminal cost changes the trade-off between short-term aggressiveness and long-term progress.
   - Why this behavior aligns with the role of terminal ingredients discussed in Lecture 5.

Module 3: Tube MPC intuition

Learning objectives

- Understand nominal versus actual trajectories under bounded disturbances.
- Visualize the idea of a "tube" that guarantees safe evolution around the nominal trajectory.
- Understand and use the ancillary feedback law $\boldsymbol{u}(k) = \boldsymbol{u}^{\mathrm{nom}}(k) - K\,\boldsymbol{e}(k)$ with $\boldsymbol{e}(k) = \boldsymbol{x}(k) - \boldsymbol{x}^{\mathrm{nom}}(k)$.
- Relate the observations to Lecture 6 (robust invariant sets, constraint tightening, robustness vs. exploration).

Background

We use the same discrete-time linear system as in Modules 1–2 and inject a bounded additive disturbance:

$$\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k + \boldsymbol{d}_k, \qquad A = \begin{bmatrix} 1 & 0.1 \\ 0 & 1 \end{bmatrix}, \; B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \; \boldsymbol{d}_k \in \mathcal{W} \subset \mathbb{R}^2. \tag{2}$$

Tube MPC separates the design into nominal and error dynamics:

$$\boldsymbol{x}_{k+1}^{\mathrm{nom}} = A\boldsymbol{x}_k^{\mathrm{nom}} + B\boldsymbol{u}_k^{\mathrm{nom}}, \qquad \boldsymbol{e}_k = \boldsymbol{x}_k - \boldsymbol{x}_k^{\mathrm{nom}}, \quad \boldsymbol{e}_{k+1} = (A - BK)\,\boldsymbol{e}_k + \boldsymbol{d}_k, \tag{3}$$

and applies the ancillary state feedback $\boldsymbol{u}(k) = \boldsymbol{u}^{\mathrm{nom}}(k) - K\boldsymbol{e}(k)$. In the provided script, the ancillary gain K is computed via a discrete-time LQR for the *error* dynamics using tunable weights $Q^{\mathrm{error}} \succeq 0$ and $R^{\mathrm{error}} \succ 0$ (function `dlqr`). The nominal input $\boldsymbol{u}_k^{\mathrm{nom}}$ is obtained from an unconstrained finite-horizon quadratic program with weights Q and R and prediction horizon N^{P}.

Tube visualization in the script: The generated plots show the nominal trajectory $\boldsymbol{x}^{\mathrm{nom}}$, the disturbed actual trajectory $\boldsymbol{x}$, and circular cross-sections, which represent a tube thickness proportional to `disturbance_bound` $\times \|K\|$ at sampled points along the nominal path (These circles provide an intuitive visualization of the tube envelope for this assignment).

What you are asked to change in the script

- Absolute disturbance bound(s) (indicated by `disturbance_bound` in the script); you may change this to, e.g., 0.1, 0.3, 0.5.

- Ancillary LQR weights for the error system: Q^{error} and R^{error} (the script includes presets; try "strong" versus "weak" feedback by changing these values).

Tasks (run → modify → analyze)

1. **Base run (tube visualization):** Run the script with the default disturbance bound (`disturbance_bound = 0.2`) and the default ancillary gain K (computed from the provided $Q^{\text{error}}, R^{\text{error}}$). Verify that the figure shows the nominal trajectory, the actual trajectory, and the tube boundary markers. Save the plot with clearly labeled nominal/actual/tube.
2. **Effect of disturbance magnitude:** Increase the disturbance bound `disturbance_bound` (e.g., $\{0.1,\ 0.3,\ 0.5\}$), re-run the script, and save the generated plots for analysis.

   Analyze: How does the apparent tube width change as the disturbance bound grows? Does the actual trajectory remain inside the displayed tube in all runs?
3. **Effect of ancillary feedback strength:** Modify Q^{error} and/or R^{error} to obtain a "strong" K (larger gains) and a "weak" K (smaller gains).

   Analyze: Explain why stronger K yields a tighter tube but may require larger control corrections, while weaker K gives a wider tube with smoother inputs.
   Relate this to the eigenvalues of $A - BK$ and the error recursion $\boldsymbol{e}_{k+1} = (A - BK)\boldsymbol{e}_k + \boldsymbol{d}_k$.
4. **Actual trajectory's behavior near the end:** Using $\boldsymbol{e}_{k+1} = (A - BK)\boldsymbol{e}_k + \boldsymbol{d}_k$ with i.i.d. bounded disturbances $\boldsymbol{d}_k$, explain why, even though $A - BK$ is Schur stable, the actual trajectory may zig-zag around the nominal trajectory near the origin.

Conclusion

In this assignment, you have:

- explored prediction and stability concepts from Lecture 4,
- implemented and tuned MPC controllers from Lecture 5,
- visualized robustness mechanisms presented in Lecture 6.

These practical exercises illustrate how mathematical control principles translate into reliable decision-making for dynamic systems.

Appendix: Linear Quadratic Regulator (LQR) gain

For the discrete-time linear system $\boldsymbol{x}(k+1) = A\boldsymbol{x}(k) + B\boldsymbol{u}(k)$, the Linear Quadratic Regulator (LQR) with cost

$$\sum_{k=0}^{\infty} \boldsymbol{e}^\top(k)\, Q^{\text{error}}\, \boldsymbol{e}(k) + \boldsymbol{u}^\top(k)\, R^{\text{error}}\, \boldsymbol{u}(k)$$

yields an optimal state feedback $\boldsymbol{u}(k) = -K\boldsymbol{x}(k)$, where $K = \left(R^{\text{error}} + B^\top PB\right)^{-1} B^\top PA$ and the symmetric positive semi-definite matrix P solves the Discrete Algebraic Riccati Equation (DARE):

$$P = Q^{\text{error}} + A^\top PA - A^\top PB\left(R^{\text{error}} + B^\top PB\right)^{-1} B^\top PA.$$

In this assignment, we apply LQR to the error dynamics $\boldsymbol{e}(k+1) = (A - BK)\,\boldsymbol{e}(k)$ with a chosen matrix pair $(Q^{\text{error}}, R^{\text{error}})$, solely to obtain a stabilizing feedback gain K that keeps the error bounded under disturbances. A full derivation of the DARE is beyond the scope of this course, but the key takeaway is that solving the LQR problem guarantees that all eigenvalues of $A - BK$ lie strictly inside the unit disk (closed-loop stability).